

夏の学校 2026

コンピュータによる計算を 数学的に調べよう

佐藤 峻

東京都立大学 大学院理学研究科

2026 年 8 月 1 日



数値実験コード
(Google Colab)

- ① コンピュータは数をどう扱う？
- ② 方程式を「だんだん解く」には？
- ③ 計算結果をどう信じる？

今日の目標

近似解を作る + 正しさを確かめる

という、数値解析の二つの役割を具体例から理解する．

参考文献：

齊藤 宣一『数値解析入門』，東京大学出版会，2012．



数値実験コード
(Google Colab)

- ① はじめに
- ② コンピュータにおける数の表現
- ③ 方程式を数値的に解く：二分法
- ④ 方程式を数値的に解く：ニュートン法
- ⑤ まとめ

方程式の例 1：連立方程式

$$\begin{cases} 2x + y = 5, \\ x - y = 1. \end{cases}$$

手で簡単に解ける。

方程式の例 2：2 次方程式

$$x^2 + 3x + 1 = 0.$$

解の公式で解ける。

(平方根の計算はここでは無視する。)

方程式の例 1：連立方程式

$$\begin{cases} 2x + y = 5, \\ x - y = 1. \end{cases}$$

手で簡単に解ける。



変数が増えると辛い

(例：変数が 10^4 個あると手で解くのは困難)

方程式の例 2：2 次方程式

$$x^2 + 3x + 1 = 0.$$

解の公式で解ける。

(平方根の計算はここでは無視する。)



多項式の次数が大きくなると辛い

(例：5 次方程式には解の公式がない)

応用上解きたい方程式は、変数が多く、個別の方程式も複雑

方程式の例 1：連立方程式

$$\begin{cases} 2x + y = 5, \\ x - y = 1. \end{cases}$$

手で簡単に解ける。



変数が増えると辛い

(例：変数が 10^4 個あると手で解くのは困難)

方程式の例 2：2 次方程式

$$x^2 + 3x + 1 = 0.$$

解の公式で解ける。

(平方根の計算はここでは無視する。)



多項式の次数が大きくなると辛い

(例：5 次方程式には解の公式がない)

応用上解きたい方程式は、変数が多く、個別の方程式も複雑

コンピュータにお願いしたいこと

大量の計算を機械的に行い、方程式の解を近似的に求める。

注：現代では、「生成 AI にお任せ」という選択肢があるが、今日はその話はしない。

コンピュータの強み

- 大量の計算を**高速**に実行可能
- 同じ手順を**何度でも**繰り返せる
- 決められた手続きを忠実に実行

良い点：速さ・量・反復

コンピュータの弱み

- ほとんどの実数は**有限桁**で近似
- 計算は**有限回**で打ち切り
- **計算法は人間が指示**する必要あり

注意点：近似・誤差・方法の選択

数学の役割

強みを活かす計算方法を設計する

+

弱みから生じる誤差を調べる

⇒ 効率的な計算・計算結果の保証

- ① はじめに
- ② コンピュータにおける数の表現
- ③ 方程式を数値的に解く：二分法
- ④ 方程式を数値的に解く：ニュートン法
- ⑤ まとめ

2 進法

$$(101)_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = (5)_{10}$$

10 進法

$$(101)_{10} = 1 \cdot 10^2 + 0 \cdot 10^1 + 1 \cdot 10^0$$

0 から 15 までの 2 進数表現：

10 進	2 進	10 進	2 進	10 進	2 進	10 進	2 進
0	$(0000)_2$	4	$(0100)_2$	8	$(1000)_2$	12	$(1100)_2$
1	$(0001)_2$	5	$(0101)_2$	9	$(1001)_2$	13	$(1101)_2$
2	$(0010)_2$	6	$(0110)_2$	10	$(1010)_2$	14	$(1110)_2$
3	$(0011)_2$	7	$(0111)_2$	11	$(1011)_2$	15	$(1111)_2$

コンピュータとの関係

コンピュータは、基本的には 0 と 1 の有限な並びで数を記録する。

n ビット (0 または 1 を n 個) で作れる並びは、全部で 2^n 通り。

n ビットの符号なし整数

$$0, 1, \dots, 2^n - 1$$

の範囲を表す。

$$(00000000)_2 = 0$$

, ...,

$$(11111111)_2 = 255$$

8 ビット (1 バイト) なら $2^8 = 256$ 通り

255 + 1 を計算しようとする表せる範囲を超えてしまい、「変なこと」が起きる。

例えば、255 + 1 = 0 となることが多い。

(符号ありの場合は、-128 から 127 の 256 通りを表すことが多い。)

整数のオーバーフローによる不具合

8/29

2010 年代に発売されたあるゲームソフトに、オーバーフローが原因と思われる不具合がある。

起きること

アイテム A は 1 つ 800 pt. 82 個買うには本来

$$800 \times 82 = 65,600 \text{ pt}$$

必要だが、なぜか 64 pt で購入できる。

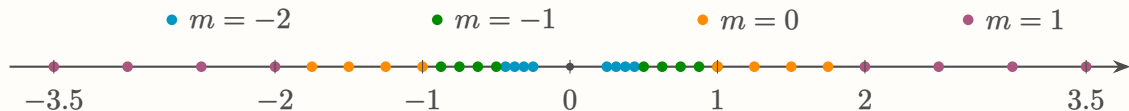
考えられる原因：16 ビットの符号なし整数

16 ビットの符号なし整数の表せる範囲：
 $0, 1, \dots, 65,535$
 $2^{16} = 65,536$ 通り

$$\begin{array}{ccc} 65,600 & = & 65,536 + 64 \\ \text{本来の価格} & & 2^{16} \end{array}$$

浮動小数点数の集合の例：

$$\{0\} \cup \left\{ \pm \left(1 + \frac{d_1}{2} + \frac{d_2}{2^2} \right) \cdot 2^m \mid d_i \in \{0, 1\}, m \in \{-2, -1, 0, 1\} \right\}$$



- 絶対値が大きいあたりは疎になっている。
- 原点の近くにも大きい隙間がある。

浮動小数点数の一般形 (よく使われる設定： $n = 52$, $L = 1022$, $U = 1023$)

$$\{0\} \cup \left\{ \pm \left(1 + \frac{d_1}{2} + \cdots + \frac{d_n}{2^n} \right) \cdot 2^m \mid d_i \in \{0, 1\}, m \in \{-L, -L+1, \dots, 0, 1, \dots, U\} \right\}$$

実際に使用されている浮動小数点数は Inf や NaN, 非正規化数 (0 付近の隙間埋め) を含むため, もう少し複雑.

前ページの浮動小数点数の集合を F とする.

$$2 = \left(1 + \frac{0}{2} + \frac{0}{2^2}\right) \cdot 2^1 \in F, \quad 0.3125 = \left(1 + \frac{0}{2} + \frac{1}{2^2}\right) \cdot 2^{-2} \in F.$$

ところが, 実数として足すと

$$2 + 0.3125 = 2.3125 \notin F.$$



有限桁での計算

最も近い浮動小数点数へ丸めると (丸めの処理を fl で表す),

$$\text{fl}(2 + 0.3125) = 2.5 \neq 2.3125.$$

表現できる数同士を足しても, 真の和を表現できるとは限らない.

次の2次方程式の小さい方の解を求める：

$$x^2 - 10^9 x + 1 = 0, \quad x_- = \frac{10^9 - \sqrt{10^{18} - 4}}{2} \approx 1.000000000000000001 \times 10^{-9}.$$

解の公式をそのまま計算

現在よく使われている浮動小数点数の演算では、途中で $\text{fl}(10^{18} - 4) = 10^{18}$ となり、

$$\text{fl}\left(\frac{10^9 - \sqrt{10^{18} - 4}}{2}\right) = 0.$$

次の2次方程式の小さい方の解を求める：

$$x^2 - 10^9 x + 1 = 0, \quad x_- = \frac{10^9 - \sqrt{10^{18} - 4}}{2} \approx 1.000000000000000001 \times 10^{-9}.$$

解の公式をそのまま計算

現在よく使われている浮動小数点数の演算では、途中で $\text{fl}(10^{18} - 4) = 10^{18}$ となり、

$$\text{fl}\left(\frac{10^9 - \sqrt{10^{18} - 4}}{2}\right) = 0.$$

分子を有理化してから計算

分子の有理化という不思議な処理をすると、

$$\text{fl}\left(\frac{2}{10^9 + \sqrt{10^{18} - 4}}\right) = 1.0 \times 10^{-9}.$$

良い近似解が得られる。

桁落ち

近い大きな数の引き算は精度がとても悪くなる。
数学的に同じ式でも、計算機上の誤差は異なり得る。

和の計算でも、大きい値と小さい値を足すとき、小さい値が無視されてしまうことがある。
→ 次の有限和のように、小さくなっていくものをたくさん足すときに問題が起こる。

$$S_n = \sum_{i=1}^n \frac{4}{i(i+1)(i+2)} = 1 - \frac{2}{(n+1)(n+2)}.$$

普通に前から順に足していくと、

n	前向きに足した結果	真値
10	0.984848484848484751	0.98484848484848484...
$\vdots$	$\vdots$	$\vdots$
416125	0.999999999990727972	0.99999999998845007...
416126	0.999999999990728083	0.99999999998845012...
416127	0.999999999990728083	0.99999999998845018...

となり、途中から更新されなくなる。

代表的な解決策が 2 つある。

和の計算でも、大きい値と小さい値を足すとき、小さい値が無視されてしまうことがある。
→ 次の有限和のように、小さくなっていくものをたくさん足すときに問題が起こる。

$$S_n = \sum_{i=1}^n \frac{4}{i(i+1)(i+2)} = 1 - \frac{2}{(n+1)(n+2)}.$$

$n = 10^6$, 真値 $S_n = 0.999999999998000005 \dots$

計算法	計算結果	真値との誤差
前向き ($i = 1 \rightarrow n$)	0.9999999999907281	7.27×10^{-12}
後ろ向き ($i = n \rightarrow 1$)	0.9999999999979999	7.28×10^{-17}
補正加算	0.9999999999979999	7.28×10^{-17}

後ろ向きに足す

小さい項から足すことで、小さい数が大きな和に埋もれにくくなる。

補正加算 (Kahan の加算法)

丸めで失われた量を補正項に記録し、次の足し算で埋め合わせる。

数学的に同じ和でも、足し方によって誤差は大きく変わる。

失われた端数を記録する

部分和を S , 補正項を c とする.
次の項 a_i を足すたびに,

$$y = a_i - c,$$

$$t = S + y,$$

$$c = (t - S) - y,$$

$$S = t$$

と更新する.

- ここでの加減算は, すべて浮動小数点演算
- c は, 今回の足し算で生じた丸め誤差
- 次回 c を差し引いてから足す

10 進 3 桁 (四捨五入) で計算する例

$$1.00 + 0.004 + 0.004$$

普通に足すと,

$$1.00 + 0.004 \rightarrow 1.00,$$

$$1.00 + 0.004 \rightarrow 1.00.$$

補正加算では, 最初に失われた 0.004 を

$$c = (1.00 - 1.00) - 0.004 = -0.004$$

により記録する. 次は実質的に

$$y = 0.004 - (-0.004) = 0.008$$

を足すため,

$$1.00 + 0.008 \rightarrow 1.01.$$

各回の小さな誤差をできるだけ捨てず, 次の計算で埋め合わせる.

ここまで：丸め誤差

有限桁で数を表し、演算するために生じる誤差を、総じて丸め誤差という。

- 桁落ちなど、丸め誤差が極端に大きくなる例を見てきた。
- 通常の倍精度計算で一回の演算に生じる誤差は、相対的にはおおむね 10^{-16} 程度のごく小さいものである。

※ オーバーフローは、表現できる範囲を超える現象であり、丸め誤差とは区別する。

この後：打ち切り誤差

コンピュータは、無限に計算を続けることもできない。

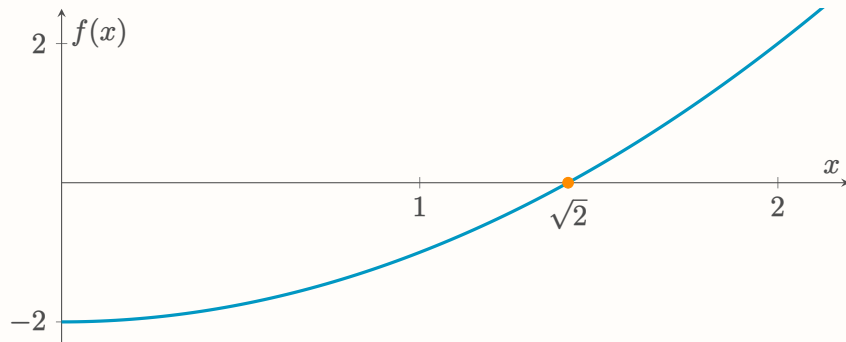
そこで、近似計算を有限回で打ち切るために生じる誤差を考える。

(「打ち切り誤差」は近似式を利用するために生じる誤差も含むが、ここでは扱わない)

これから扱う計算法では、通常は打ち切り誤差の方が大きいことが多い。

- ① はじめに
- ② コンピュータにおける数の表現
- ③ 方程式を数値的に解く：二分法
- ④ 方程式を数値的に解く：ニュートン法
- ⑤ まとめ

$$f(x) = x^2 - 2 = 0 \quad \Longleftrightarrow \quad x = \pm\sqrt{2}.$$



解の1つである $\sqrt{2}$ の近似値を求める代表的な方法である二分法を紹介する.

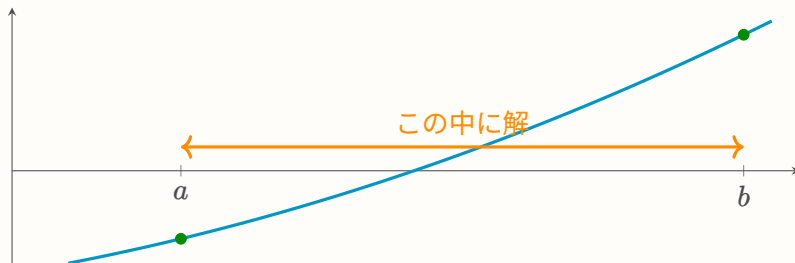
中間値の定理

閉区間 $[a, b]$ 上で連続な関数 f について (a, b は $a < b$ を満たす定数),
 $f(a)f(b) < 0$ を満たすとき ^a, $f(x) = 0$ を満たす点が开区間 (a, b) に 1 つ (以上) 存在する.

^a要するに, $f(a)$ と $f(b)$ の符号が異なるとき

(a, b) : $a < x < b$ を満たす範囲 (両端を含まない, 开区間)

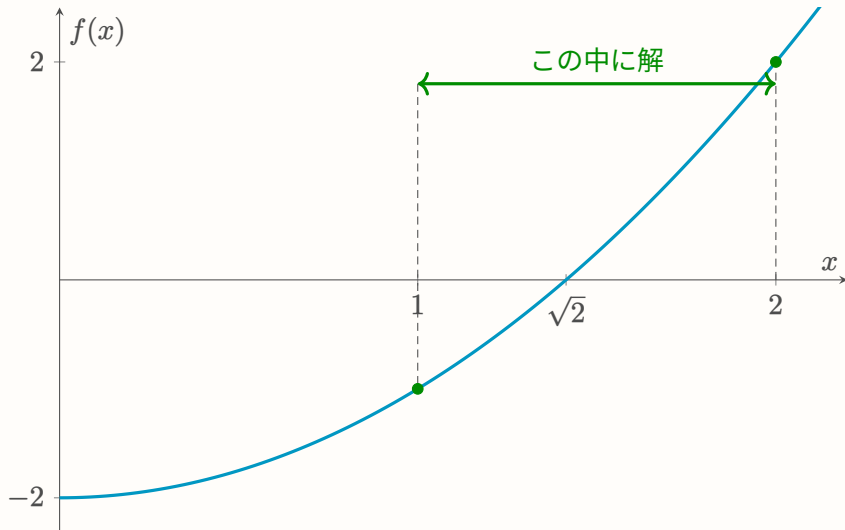
$[a, b]$: $a \leq x \leq b$ を満たす範囲 (両端を含む, 闭区間)



二分法のイメージ：区間を半分ずつ狭める

18/29

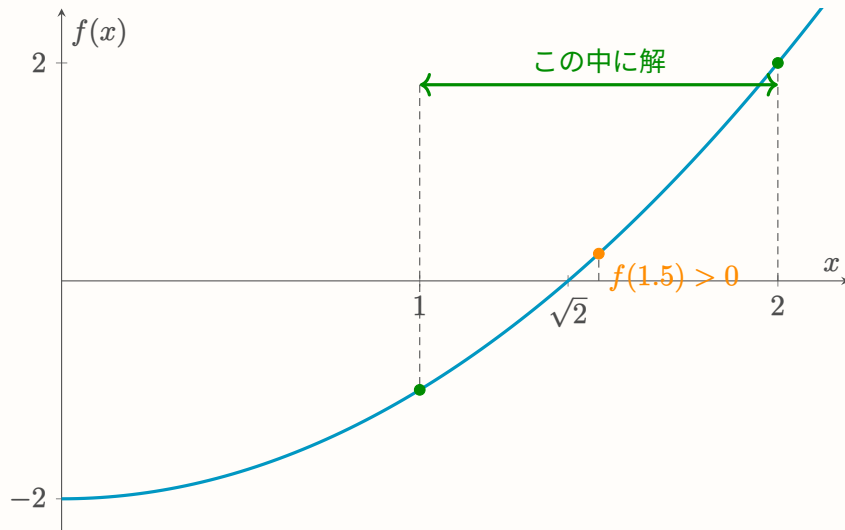
$f(x) = x^2 - 2$ について, $f(1) < 0, f(2) > 0$ より, $(1, 2)$ に解がある.



二分法のイメージ：区間を半分ずつ狭める

18/29

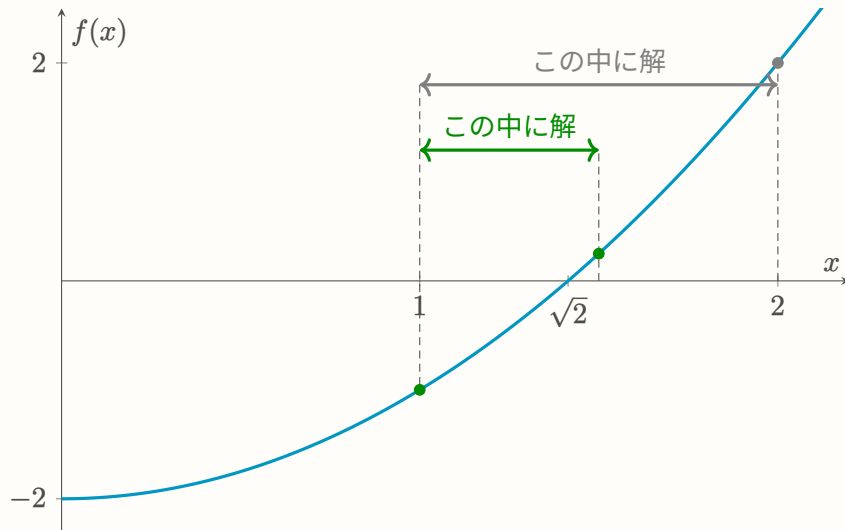
(1, 2) の中点 1.5 では $f(1.5) > 0 \rightarrow$ 次は (1, 1.5) を調べる。



二分法のイメージ：区間を半分ずつ狭める

18/29

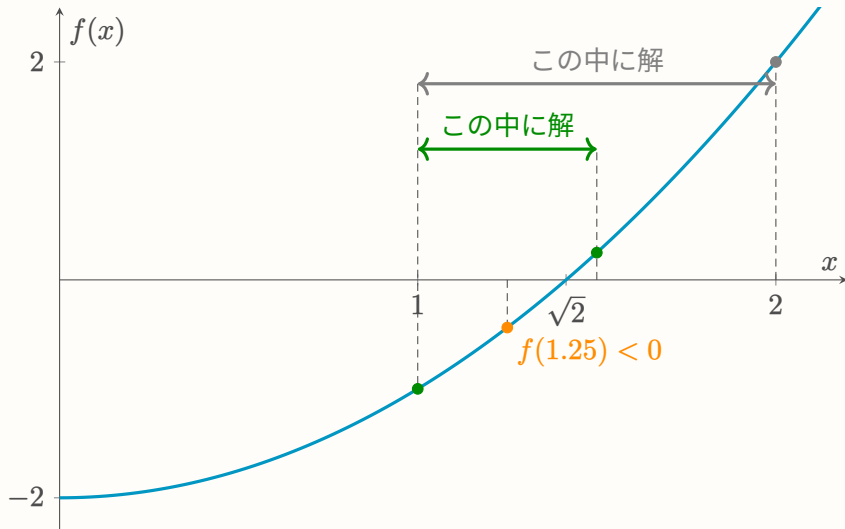
(1, 2) の中点 1.5 では $f(1.5) > 0 \rightarrow$ 次は (1, 1.5) を調べる。



二分法のイメージ：区間を半分ずつ狭める

18/29

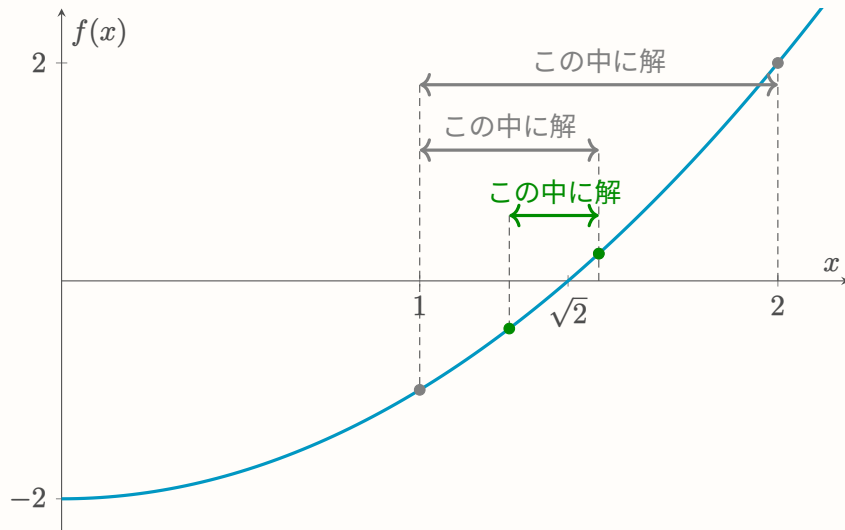
(1, 1.5) の中点 1.25 では $f(1.25) < 0 \rightarrow$ 次は (1.25, 1.5) を調べる.



二分法のイメージ：区間を半分ずつ狭める

18/29

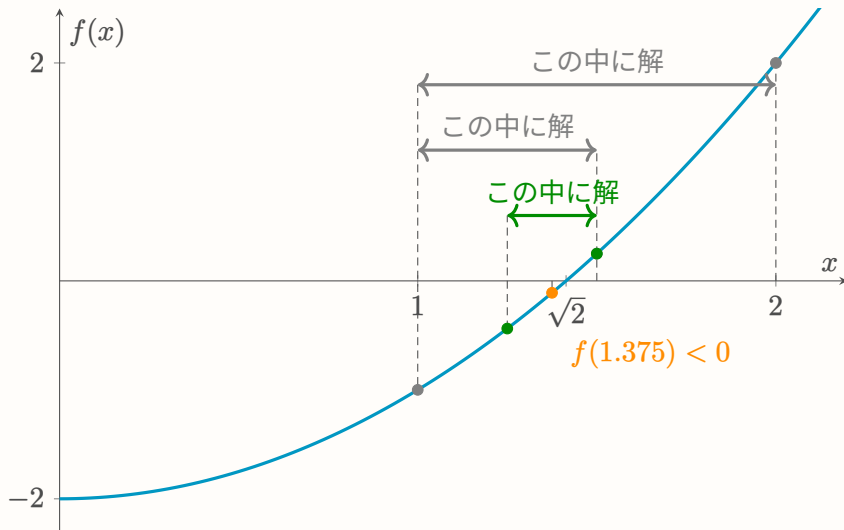
(1, 1.5) の中点 1.25 では $f(1.25) < 0 \rightarrow$ 次は (1.25, 1.5) を調べる。



二分法のイメージ：区間を半分ずつ狭める

18/29

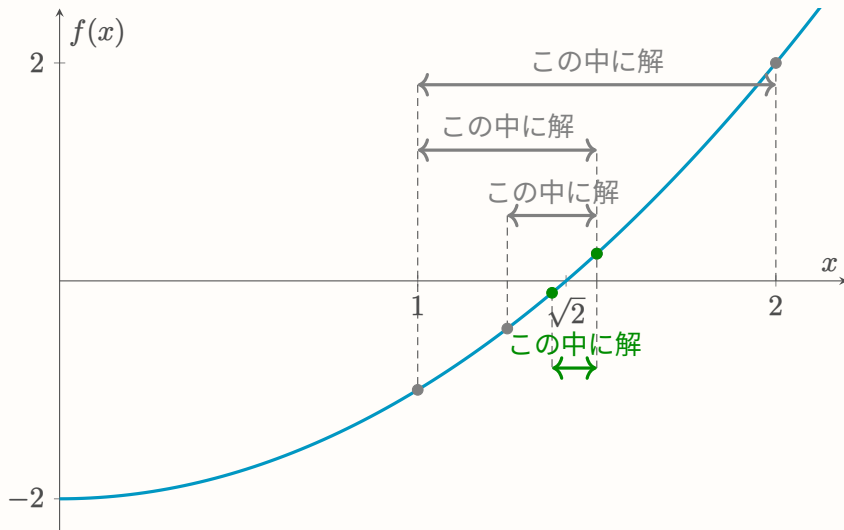
(1.25, 1.5) の中点 1.375 では $f(1.375) < 0 \rightarrow$ 次は (1.375, 1.5) を調べる.



二分法のイメージ：区間を半分ずつ狭める

18/29

(1.25, 1.5) の中点 1.375 では $f(1.375) < 0 \rightarrow$ 次は (1.375, 1.5) を調べる.



$\rightarrow$ 15 回繰り返すと, $[1.4141845703125, 1.414215087890625]$ に解があることが分かる.

入力： 連続な関数 f ，符号の異なる両端 a, b ，反復回数 N ．ただし， $f(a)f(b) < 0$ ．

N 回繰り返す

- ① 中点を計算する： $m \leftarrow \frac{a+b}{2}$
- ② $f(m) = 0$ ならば， m を解として終了する．
- ③ そうでなければ，符号が変わる半分を残す：
 - $f(a)f(m) < 0$ のとき， b に m を代入
 - $f(m)f(b) < 0$ のとき， a に m を代入

出力： 最後に残った区間 (a, b) の中点 $\frac{a+b}{2}$ を近似解とする．

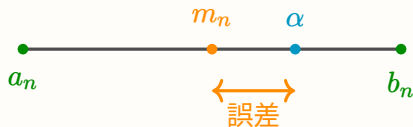
各反復で $f(a_n)f(b_n) < 0$ を保つ $\implies$ 中間値の定理より解 $\alpha \in (a_n, b_n)$ が存在.

最初の区間の幅を $b_0 - a_0$ とすると, n 回後の幅は

$$b_n - a_n = \frac{b_0 - a_0}{2^n}.$$

中点 m_n を答えにすれば, 真の解 α との差は

$$|m_n - \alpha| \leq \frac{b_n - a_n}{2} = \frac{b_0 - a_0}{2^{n+1}}.$$



例：最初の区間が $(1, 2)$

15 回後には

$$b_{15} - a_{15} = 2^{-15} \approx 3.05 \times 10^{-5}, \quad |m_{15} - \sqrt{2}| \leq 2^{-16} \approx 1.53 \times 10^{-5}.$$

真の解を知らなくても, 反復回数から誤差を保証できる.

注：ここでは丸め誤差は無視している.

- ① はじめに
- ② コンピュータにおける数の表現
- ③ 方程式を数値的に解く：二分法
- ④ 方程式を数値的に解く：ニュートン法
- ⑤ まとめ

ニュートン法のイメージ：接線で次の点を予想する

22/29

現在の近似値を x_n とする.

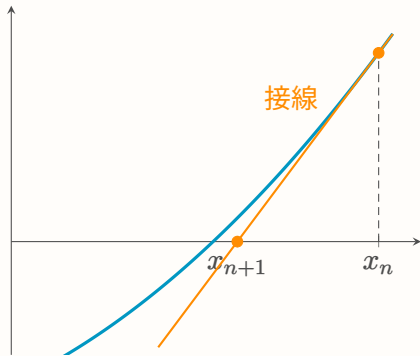
点 $(x_n, f(x_n))$ での接線は

$$y = f(x_n) + f'(x_n)(x - x_n).$$

この接線と x 軸の交点を, 次の近似値 x_{n+1} とする.

更新式

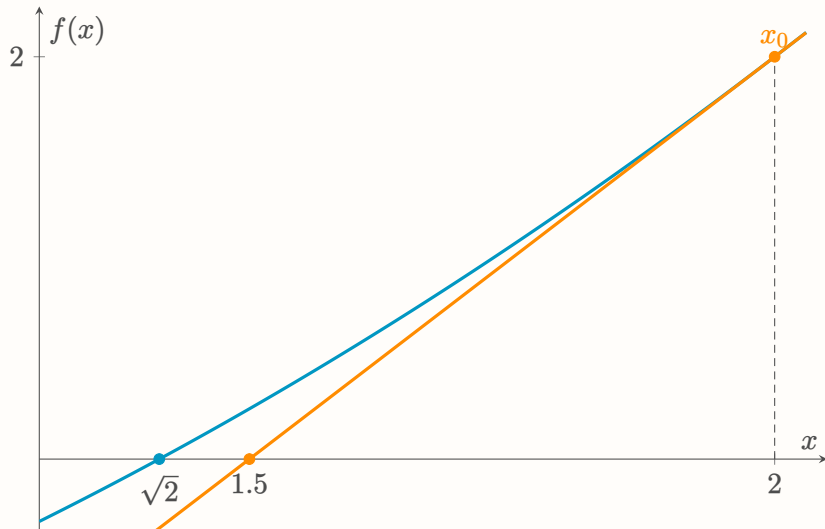
$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$



ニュートン法で $\sqrt{2}$ に近づく

23/29

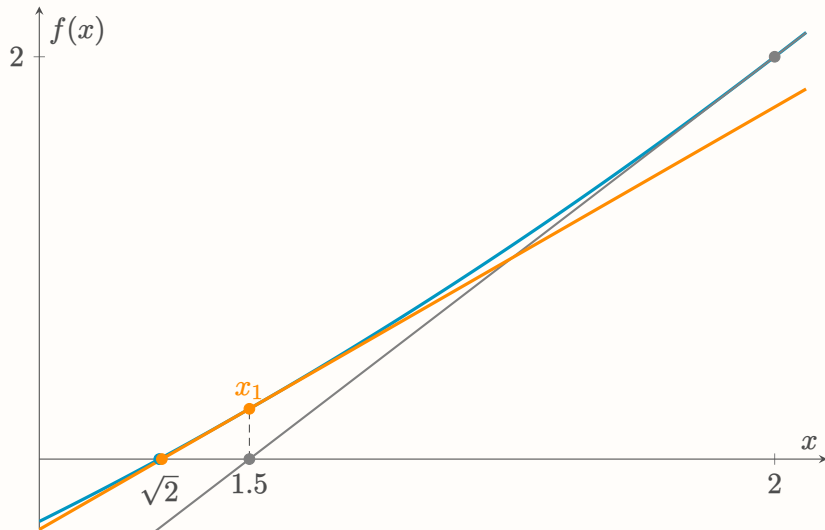
$x_0 = 2$ での接線は $y = 4x - 6$. x 軸との交点は $x_1 = 1.5$.



ニュートン法で $\sqrt{2}$ に近づく

23/29

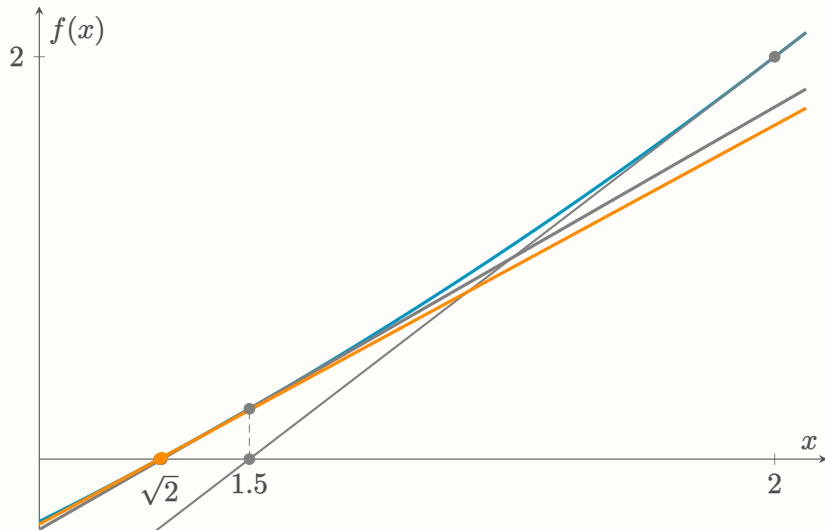
$x_1 = 1.5$ での接線は $y = 3x - \frac{17}{4}$. x 軸との交点は $x_2 = \frac{17}{12} = 1.416666\dots$



ニュートン法で $\sqrt{2}$ に近づく

23/29

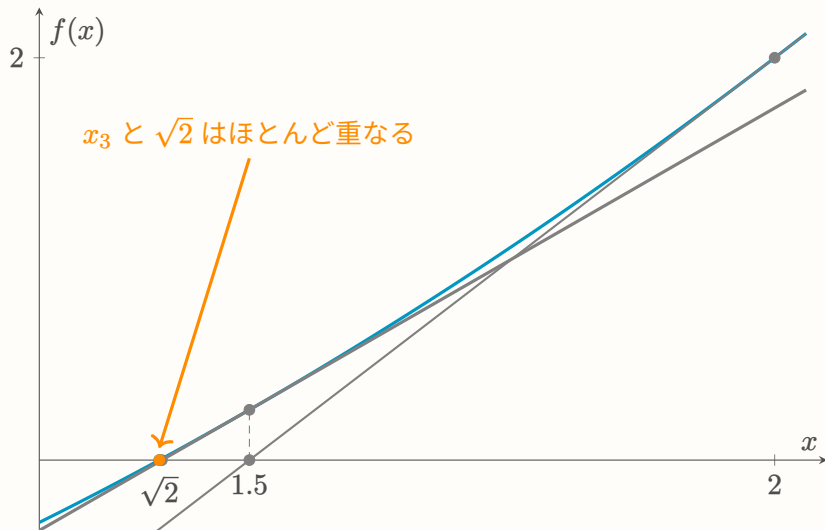
$$x_2 = \frac{17}{12} \text{ での接線は } y = \frac{17}{6}x - \frac{577}{144} \cdot x_3 = \frac{577}{408} = 1.414215 \dots$$



ニュートン法で $\sqrt{2}$ に近づく

23/29

わずか3回で、 $\sqrt{2} = 1.414213\dots$ の小数第5位まで一致する。



入力： 関数 f ，導関数 f' ，初期値 x_0 ，反復回数 N ．

$n = 0, \dots, N - 1$ について実行する

- ① $f'(x_n) = 0$ ならば，このままでは計算できないので終了する．
- ② 接線と x 軸の交点を計算する：

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

出力： 最後に得られた x_N を近似解とする．

区間ではなく，一つの近似値を更新していく．

$f(x) = x^2 - 2$ の場合，更新式は $x_{n+1} = \frac{1}{2} \left(x_n + \frac{2}{x_n} \right)$.

真の解 $\sqrt{2}$ との差を計算すると，

$$\begin{aligned} x_{n+1} - \sqrt{2} &= \frac{x_n^2 + 2 - 2\sqrt{2}x_n}{2x_n} \\ &= \frac{(x_n - \sqrt{2})^2}{2x_n}. \end{aligned}$$

解の近くでは

誤差 $e_n = |x_n - \sqrt{2}|$ が小さければ，

$$e_{n+1} \approx \frac{e_n^2}{2\sqrt{2}}.$$

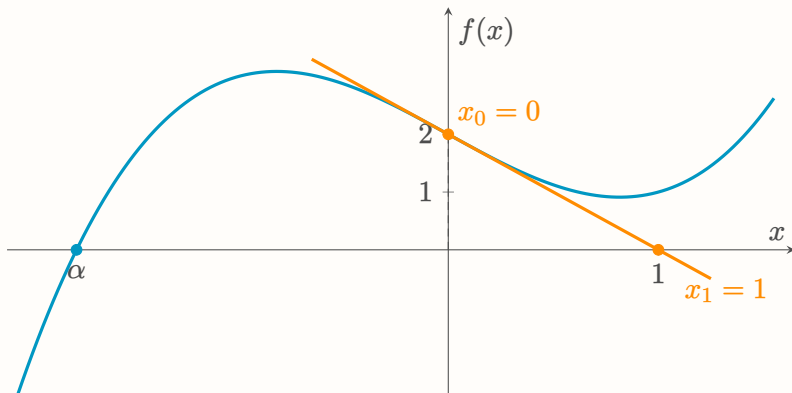
誤差がおおよそ **2 乗** になるので，正しい桁数が急速に増える．

※ この速さは，解の近くから始め， $f'(\sqrt{2}) \neq 0$ であることに支えられている．

例： $f(x) = x^3 - 2x + 2$, $x_0 = 0$

$$x_1 = 1, \quad x_2 = 0, \quad x_3 = 1, \quad \dots$$

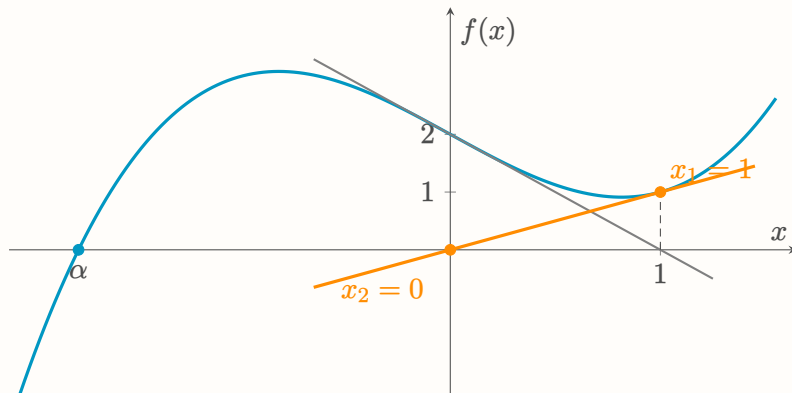
二つの点を行き来して、解に近づかない。



例： $f(x) = x^3 - 2x + 2$, $x_0 = 0$

$$x_1 = 1, \quad x_2 = 0, \quad x_3 = 1, \quad \dots$$

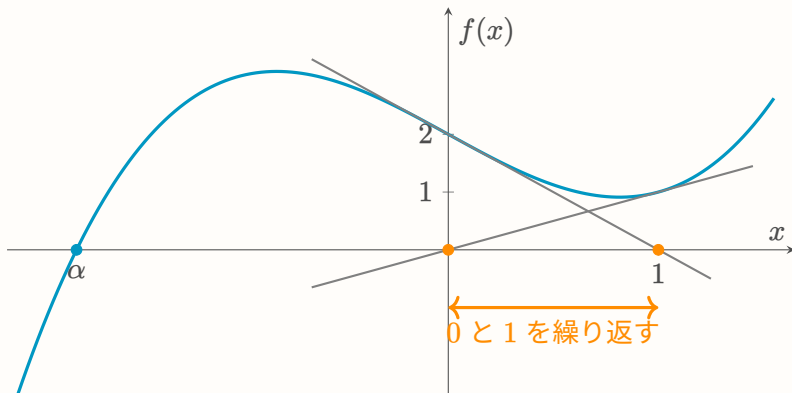
二つの点を行き来して、解に近づかない。



例： $f(x) = x^3 - 2x + 2$, $x_0 = 0$

$$x_1 = 1, \quad x_2 = 0, \quad x_3 = 1, \quad \dots$$

二つの点を行き来して、解に近づかない。



	二分法	ニュートン法
速さ	一定の割合で進む	うまくいけば非常に速い
必要な情報	関数値の符号	関数値と導関数
出発条件	解を挟む区間	一つの初期値
確実性	仮定の下で確実	失敗することがある
誤差の保証	区間幅から容易	別の議論が必要

数値計算法を選ぶ視点

「速いか」だけでなく、必要な情報、失敗の可能性、誤差の評価まで考える。

- ① はじめに
- ② コンピュータにおける数の表現
- ③ 方程式を数値的に解く：二分法
- ④ 方程式を数値的に解く：ニュートン法
- ⑤ まとめ

- ① コンピュータは，多くの数値計算で有限桁の数を使い，有限回の計算を行う．
- ② 二分法は確実に誤差を評価しやすく，ニュートン法はうまくいけば速い．
- ③ 近似解を作る方法と，その信頼性を調べるのが数値解析である．

持ち帰ってほしいこと

計算結果を信じるためにも，数学が必要である．

Appendix

定理：ニュートン法の収束

- 関数 f は開区間 (a, b) で 2 回連続的微分可能
- $f(\alpha) = 0$ を満たす α が (a, b) にただ一つ存在
- $f'(\alpha) \neq 0$

このとき, α に十分近いすべての x_0 に対して, ニュートン法は数列 $\{x_n\}$ を生成し, この数列は α に 2 次収束する. ここで, 2 次収束とは, すべての $n \geq n_0$ で

$$|x_n - \alpha| \leq Cr^{2^{n-n_0}}$$

が成立するような $n_0 > 0, C > 0, 0 < r < 1$ が存在することをいう.